

SPECTRON AgriAssist Architecture

Updated Architecture Document - Alert System & Role-Based Access Control (Owner/Viewer)

Added

Executive Summary

SPECTRON AgriAssist is a domain-specific IoT platform for Sri Lankan agriculture, focused on monitoring crops and providing AI-driven recommendations. The system combines modular hardware (controllers, sensor bases, sensor modules), cloud microservices, real-time weather data, and a Retrieval-Augmented Generation (RAG) AI pipeline (using the DeepSeek LLM) to advise farmers on irrigation, fertilization, pests, and more. This Architecture Document defines a comprehensive design for frontend and backend components, data models, AI integration, hardware provisioning, APIs, deployment, security, and a roadmap for implementation.

This revision adds two previously missing subsystems: (1) a dedicated Alert System that generates and delivers threshold- and event-based notifications from weather, sensor, hardware, and recommendation events; and (2) a full Role-Based Access Control model supporting farm-level collaboration, where a farm Owner can invite one or more Viewers with read-only access to that specific farm.

This connectivity update also adds a resilient multi-network strategy for field controllers: Wi-Fi is preferred when a configured network and the cloud endpoint are reachable; the existing 2G/GPRS SIM module remains an automatic fallback; and durable local store-and-forward protects sensor data when neither path is available. Wi-Fi provisioning is offered as an optional step during controller onboarding and remains editable from the Hardware configuration page. Credentials are transferred locally through a secured BLE provisioning session or protected SoftAP fallback and are encrypted on the controller rather than stored as plaintext in the cloud. [NEW - CONNECTIVITY]

Table of Contents

- 1. Introduction
- 2. Requirements (2.1 Functional, 2.2 Non-Functional)
- 3. Domain Model (UML & ER)
- 4. Frontend Design
- 5. Backend Design (Microservices)
- 6. Database Design (PostgreSQL Schema & ER Diagram)
- 7. AI Recommendation Engine (RAG Pipeline)
- 8. Hardware Integration
- 9. API Specification (REST Endpoints)
- 10. Deployment Architecture
- 11. Security (Auth, RBAC, Encryption, Data Privacy)
- 12. Alert System [NEW]
- 13. Role-Based Access & Farm Collaboration [NEW]
- 14. Testing, QA, and Rollout Plan
- 15. Maintenance & Future Work
- 16. Implementation Roadmap
- 17. References
- 18. Resilient Controller Connectivity (Wi-Fi + Cellular Fallback) [NEW]

1. Introduction

Modern Sri Lankan farms need smart advice on irrigation, fertilization, pest control, etc., tailored to local conditions. SPECTRON AgriAssist fills this gap with an IoT + AI platform. It replaces generic "controller/dashboard" UIs with a farm-centric workflow: farmers manage Farms and Fields (planted with specific crops/varieties), view Weather, monitor Crop Health, and get AI-driven recommendations. For example, rice occupies ~29% of Sri Lanka's cultivated land (~1.317M ha), yielding 3.68M tons/year. It provides ~45% of daily calories and 40% of protein for the average Sri Lankan. AgriAssist will tap into this domain knowledge (from RRDl, DOA, IRRI, FAO, CABI, etc.) and local sensor data to help maintain productivity and food security.

Two collaboration and safety needs are central to real-world adoption: farmers must be alerted promptly to time-sensitive risks (weather, pest, hardware failure), and farm data must be shareable with trusted collaborators (family members, agronomists, farm workers) without giving away full control. Sections 12 and 13 formalize both.

2. Requirements

2.1 Functional Requirements

The system shall:

- User Management: Allow farmers to register/login, manage profiles, and have role-based access. Roles are now scoped per farm (Owner, Viewer) rather than global, in addition to the platform-level Admin role.

- Farm & Field Management: Users can create/edit one or more Farms. Each farm can have multiple Fields/Plots. Each field can have one or more Crop Instances (if multiple crops share the plot).

- Farm Collaboration [NEW]: The Owner of a farm can invite additional users as Viewers, granting them

read-only access to that farm's fields, monitoring data, weather, recommendations, and history. Owners can revoke Viewer access at any time.

- Crop Monitoring: For each Crop Instance (Crop + Variety + Planting date + Growth Stage), track sensor data (e.g. temp, humidity, soil moisture, pH) and live weather. Display health metrics (e.g. water stress, disease risk).

- Hardware Management: Support modular IoT devices. Farmers can register controllers (gateways) and assign them to fields. Each controller connects to one or more Sensor Bases, which in turn have interchangeable sensor modules (temp, moisture, light, etc.). UI shall allow adding/removing sensors in each base.

- Data Ingestion: Continuously ingest sensor data via MQTT from controllers; store in time-series DB or Kafka for processing. Retrieve weather forecasts via an external API for each field's location.

- AI Recommendations: Based on current crop, growth stage, weather, and sensor data, generate actions (e.g. "delay irrigation", "apply X fertilizer"). Users can ask natural-language queries. Each recommendation must reference evidence from the knowledge base.

- Knowledge Base: Maintain a curated agri-knowledge DB (crop info, pest/disease profiles, fertilizer guidelines, local DOA publications, etc.). Used for RAG retrieval.

- Feedback & History: Farmers can rate and comment on recommendations. The system stores all recommendations, their outcomes, and user feedback for analytics.

- Analytics Dashboard: Aggregate stats (avg. recommendation ratings, common issues, sensor trends).

- Alerts [NEW / EXPANDED]: The system shall generate structured, persisted alerts for extreme weather,

pest/disease risk, sensor anomalies (out-of-range or missing readings), and hardware faults (gateway offline). Alerts shall be delivered to all users with access to the affected farm (Owner and Viewers), shown on the dashboard notification panel, and shall support read/unread/dismissed states per recipient.

- Localization: Support Sinhala, Tamil, English UI (text must be translatable).

- Accessibility: UI must follow WCAG guidelines (contrast, keyboard nav, alt text, etc.).

2.2 Non-Functional Requirements

- Scalability: Must scale to thousands of farms and high-frequency sensor streams. Microservices architecture (see Sect. 5) allows horizontal scaling.

- Reliability & Fault Tolerance: Use Kubernetes (or cloud containers) for auto-restart. Use MQTT QoS and retry for IoT data. Store data durably.

- Performance: Real-time (near-real-time) updates on monitoring dashboards. Generate AI recommendations in seconds. Alerts must be raised within seconds of the triggering event.

- Security: Authenticate via JWT/OAuth2. Use TLS for all communications (REST, MQTT). Encrypt sensitive data (user credentials, API keys). Implement fine-grained RBAC at the farm level: Owner vs. Viewer, plus platform-level Admin (see Sect. 13).

- Maintainability: Modular code (Go microservices), Docker, CI/CD, automated tests. Logged and monitored (Prometheus/Grafana).

- Usability: Intuitive UI for farmers; minimal training required. Clear labels, tooltips, and contextual help (especially in the Knowledge Center).

2.3 Connectivity Requirements [NEW]

- Preferred Connectivity: A controller shall prefer a configured Wi-Fi network when Wi-Fi is available and the cloud endpoint is reachable.
- Automatic Cellular Fallback: If Wi-Fi association, internet access, DNS, TLS, MQTT, or cloud health checks repeatedly fail, the controller shall automatically use the existing 2G/GPRS SIM connection.
- Stable Failover: The firmware shall use retry thresholds, exponential backoff, minimum dwell time, and recovery checks to prevent rapid switching between Wi-Fi and cellular.
- Offline Continuity: If both Wi-Fi and cellular are unavailable, readings and critical events shall be stored in a durable local queue and uploaded later with their original timestamps and sequence numbers.
- Optional Provisioning: Wi-Fi setup shall be offered during Add Controller but shall not block registration. Owners may skip it and use cellular, then add or change Wi-Fi from the Configure Connectivity page.
- Secure Credential Transfer: Credentials shall be provisioned locally using a secured BLE session where supported, with a protected controller SoftAP as fallback. Plaintext Wi-Fi passwords shall not be stored in the cloud or returned by an API.
- Multiple Networks: A controller may store a small ordered list of trusted networks, such as farm Wi-Fi, an owner hotspot, and a maintenance network. Unsecured public networks shall not be joined automatically.
- Traffic Prioritization: Critical alerts and health events shall be transmitted immediately; routine telemetry may be batched; backlog synchronization shall be rate-limited; firmware downloads should prefer Wi-Fi.
- Connectivity Visibility: The platform shall show active transport, signal quality, last successful cloud contact, queued-record count, and fallback state.

3. Domain Model

The domain entities include User, Farm, FarmCollaborator [NEW], Field, CropInstance, Variety, GrowthStage, Gateway (Controller), SensorBase, SensorModule, WeatherData, Recommendation, Feedback, KnowledgeBaseEntry, and Alert [NEW]. Relationships:

- A User can be linked to many Farms through FarmCollaborator records, each carrying a role: owner or viewer.

A Farm has exactly one Owner (the creator, by default) and zero or more Viewers.

- A Farm has many Fields/Plots. Each Field is at a specific location and area.
- Each Field may grow one or multiple crops (e.g. mixed planting); for each planted crop we have a CropInstance record linking Crop + Variety + planting and expected harvest dates + current GrowthStage.

- A Field is monitored by exactly one Gateway device (the farm's IoT controller). Each Gateway attaches to one

or more SensorBase units. Each SensorBase has slots for SensorModules (e.g. temperature, humidity, soil moisture, pH sensors).

- SensorModule data and external WeatherData (fetched per-field location) are tied to a specific

Field/CropInstance context.

- The KnowledgeBase stores agricultural info which is retrieved by the AI when generating a Recommendation.
- Each Recommendation record is linked to the Field/Crop it concerns and stores the suggested action,

rationale, timestamp, and KnowledgeBaseEntry references used. Farmers provide Feedback on each recommendation.

- An Alert is generated by a source service (Weather, Sensor Data, Hardware, or Recommendation) and is

linked to a Farm and optionally a Field. It is fanned out to every User with an active FarmCollaborator record for that farm, and each recipient tracks their own read/dismissed state.

(An abstract class diagram would show classes connected by "owns/has" arrows, composition arrows for hardware, and a new many-to-many arrow between User and Farm via FarmCollaborator, plus a one-to-many arrow from Farm to Alert and Alert to AlertRecipient.)

On the database side (see Sect. 6), tables map directly to these entities with foreign keys: e.g. fields(farm_id), crop_instances(field_id, variety_id, stage_id), sensor_readings(field_id, sensor_type, timestamp, value), farm_collaborators(farm_id, user_id, role), alerts(farm_id, field_id), alert_recipients(alert_id, user_id).

3.1 Connectivity Domain Additions [NEW]

The connectivity model extends Gateway with network and synchronization state. Supporting entities/configurations include GatewayConnectivityProfile, SavedWiFiNetwork, ConnectivityEvent, and OfflineQueueState. A Gateway has one active connectivity profile; may hold multiple encrypted saved Wi-Fi networks in priority order; reports its current transport (wifi, cellular, offline); and reports queue depth and last synchronization status. The cloud stores network identifiers and status metadata only where required for management; the Wi-Fi secret remains encrypted on the physical controller.

4. Frontend Design

The UI is entirely agriculture-centric. Controllers/sensors are hidden in Hardware pages; primary navigation is by farms, crops, recommendations.

Navigation: Top menu with icons/tabs: Dashboard, My Farms, My Crops, Crop Health, AI Advisor, Weather, Hardware, Knowledge Center, History, Alerts [NEW], Settings/Profile.

4.1 Layout & Styles

- Responsive web app (works on desktop and tablets).
- Color-coded status indicators (green/yellow/red) for quick health alerts.
- Consistent header with user info and notifications (e.g. upcoming rain, pest alerts, unread alert badge count

[NEW]).

- Use accessible fonts and color schemes (e.g. high contrast text on fields).

4.2 Pages (updated)

Login / Registration:

- Login: Fields for email and password, "Forgot Password" link, language switch. Basic validation.
- Registration: Collect user name, email, password, confirm password. On success, send verification email.

Dashboard (Home):

- Shows "My Farm(s)" overview, filtered to farms the user owns or has been added to as a Viewer [UPDATED].
- Displays current weather summary, active crop and stage, latest recommendation, and top alerts.

- Quick stats: e.g. "Total Farms: 2", "Active Alerts: 1".

• Notification panel showing unread Alerts, ordered by severity then recency [NEW].

My Farms:

- Lists farms the user owns AND farms shared with them as a Viewer, with a role badge ("Owner" / "Viewer")

next to each [UPDATED].

- Buttons to Add Farm, View Farm. "Add Farm" is only available to create new farms the user will own.

Farm Details:

- Displays farm info and list of fields/plots.
- Each field card shows crop, area, sensor status, latest soil moisture reading, and health indicator.

• Owners see "Add Field", "Manage Hardware", and "Manage Access" [NEW] actions. Viewers see the same data in read-only form with these actions hidden.

Field (Plot) Details, Crop Instance Page, Crop Health Dashboard, Weather Page, AI Advisor, Knowledge Center, Hardware & Sensors, Recommendation History, Feedback UI, Analytics Page: unchanged from the base design, with the addition that all write actions (assign crop, edit hardware, submit feedback, trigger recommendation) are gated to Owner role; Viewers get a read-only rendering of the same pages.

Manage Access Page [NEW] (under Farm Details, Owner-only):

- Lists current collaborators for the farm with name, email, role, and date added.
- "Invite Viewer" form: email input, sends an invitation (existing users are added directly; new emails receive a signup + auto-link invite).
- "Remove" action next to each Viewer to revoke access immediately.

Alerts Page [NEW]:

- Full list of alerts across all accessible farms, filterable by farm, severity, type, and status (unread/read/dismissed).
- Clicking an alert deep-links to the relevant Field, Crop Instance, or Hardware page.
- Mark-as-read and dismiss actions, plus "Mark all as read".

Profile & Settings:

- User personal data, contact info, password change.
- Notification preferences (e.g. email when new alert or recommendation is ready) [UPDATED].
- API keys for sensor integration, if needed.

4.3 Controller Connectivity Setup [NEW]

Add Controller remains the primary onboarding workflow and is not replaced. After the controller is claimed and assigned to a farm/field, an optional Configure Connectivity step is shown. The Owner can scan available networks, enter a password, test the connection, or select "Skip for now - use cellular." The controller is considered successfully added even when Wi-Fi setup is skipped.

The Hardware > Controller > Configure Connectivity page remains available after onboarding and allows the Owner to add, reprioritize, update, test, or remove trusted Wi-Fi networks; view current transport, Wi-Fi RSSI, cellular signal, last cloud contact, and queued readings; enable Automatic mode; and temporarily force a transport for diagnostics. Viewers may see connection status but cannot view or change credentials. Password fields are write-only and are never redisplayed.

Recommended onboarding sequence: (1) scan QR code or enter serial/proof-of-possession code; (2) assign farm and field; (3) optionally provision Wi-Fi through BLE or controller SoftAP; (4) controller verifies local association, internet access, TLS/MQTT connection, and cloud acknowledgement; (5) UI confirms "Wi-Fi connected / cellular fallback ready" or "Cellular active / Wi-Fi not configured"; (6) finish setup.

5. Backend Design

The backend is a set of microservices (in Go) communicating via REST and message queues.

5.1 Service Catalog (updated)

- Auth/User Service: Manages user accounts, authentication (JWT tokens). JWT now carries only identity

(user_id) and platform-level flag (Admin or not); farm-level role is resolved per-request against FarmCollaborator, not embedded in the token [UPDATED].

- Farm Service: CRUD for farms. Endpoints: GET/POST/PUT/DELETE /api/farms. On farm creation, automatically creates a FarmCollaborator record with role=owner for the creator.

- Collaboration Service [NEW]: Manages FarmCollaborator records - inviting, listing, and removing Viewers

for a farm. Enforces that only the farm's Owner can call invite/remove endpoints, and that a farm always retains exactly one Owner.

- Field Service: CRUD for fields/plots.
- Crop Service: Manages crop types and varieties.
- CropInstance Service: Tracks specific plantings.
- Hardware Service: Manages controllers/gateways, sensor bases, and sensor slots.
- Sensor Data Service: Handles incoming sensor readings via MQTT -> Kafka -> time-series storage. Emits

"out_of_range" or "no_data_timeout" events consumed by the Alert Service [NEW HOOK].

- Weather Service: Periodically calls an external Weather API. Emits "severe_weather" events (e.g. rain

probability, extreme temperature/wind thresholds) consumed by the Alert Service [NEW HOOK].

- Knowledge Service: Maintains the RAG index and retrieval.
- Recommendation Service: Orchestrates AI recommendation generation. Emits "high_risk_recommendation"

events (e.g. disease/pest risk above a confidence threshold) consumed by the Alert Service [NEW HOOK].

- AI/DeepSeek Service: Thin wrapper over the LLM.
- Feedback Service: Receives user ratings/comments.
- Analytics Service: Aggregates data and KPIs.

- Alert Service [NEW]: Central service that receives trigger events from Weather, Sensor Data, Hardware, and

Recommendation services (via Kafka), applies alert rules (thresholds, deduplication, severity classification), persists Alert records, and fans them out to all current FarmCollaborators of the affected farm by creating AlertRecipient rows. Exposes read/dismiss endpoints (see Sect. 9).

Inter-Service Communication: Most services expose REST endpoints. MQTT handles raw IoT device communication; Kafka is the backbone for internal events (sensor data, recommendation events, and now

alert-trigger events [UPDATED]).

5.2 Sequence Example - Alert Generation [NEW]

1. Sensor Data Service detects a soil-moisture reading outside the configured safe range for the crop's growth stage.
2. It publishes a "sensor_out_of_range" event to Kafka with field_id, sensor_type, value, and threshold.
3. Alert Service consumes the event, resolves field_id to farm_id, classifies severity, and checks for an

existing open alert of the same type to avoid duplicates.

- 4. Alert Service writes an alerts row and, for every user in farm_collaborators for that farm, writes an alert_recipients row.
- 5. Dashboard/notification panel polls or subscribes (via WebSocket/SSE) for new alerts and updates the badge count for each affected user.

5.3 Sequence Example - Adding a Viewer [NEW]

- 1. Owner opens "Manage Access" on a farm and submits an email.
- 2. Collaboration Service verifies the caller is the farm's Owner (checked against farm_collaborators, not JWT role).
- 3. If the email matches an existing user, a farm_collaborators row (role=viewer) is created immediately and the user gains read access on next login/refresh.
- 4. If no user exists with that email, an invitation record is created and an email is sent; on signup, the invite is redeemed and the row is created.
- 5. The new Viewer appears in "My Farms" for that user, and in the collaborator list for the Owner.

Fault Tolerance: Each microservice runs in Docker containers (Kubernetes). Critical services (Auth, Recommendation, Alert [UPDATED]) are replicated. Use circuit breakers when calling external AI or weather APIs.

5.4 Connectivity Responsibilities [NEW]

- Hardware Service: Owns connectivity configuration metadata and status, exposes provisioning-session and connectivity-status APIs, receives controller health reports, and emits connectivity events.
 - Sensor Data Service: Accepts idempotent batch uploads from the offline queue and preserves original measurement timestamps and sequence numbers.
 - Alert Service: Consumes sustained connectivity failures and offline-buffer risk events. Brief transport changes are recorded as events but do not necessarily become user-facing alerts.
- Device Firmware: Implements the actual network state machine. It attempts trusted Wi-Fi first, verifies end-to-end cloud reachability, falls back to cellular after a configurable failure threshold, periodically tests Wi-Fi while on cellular, and returns only after Wi-Fi is stable. When both paths fail, it writes to durable local storage.

Recommended state model: WIFI_CONNECTING -> WIFI_ONLINE -> CELLULAR_CONNECTING -> CELLULAR_ONLINE -> OFFLINE_BUFFERING. Transitions use bounded retries and exponential backoff. Suggested defaults are 3-5 failed Wi-Fi attempts before fallback, 5-15 minutes minimum cellular dwell time, and a successful cloud health check before returning to Wi-Fi. These are configurable defaults, not hard-coded agronomic rules.

Controller telemetry should include gateway_id, active_transport, wifi_signal_dbm, cellular_signal, last_cloud_success_at, queued_records, queue_capacity_percent, firmware_version, and connectivity_reason.

6. Database Design

We use PostgreSQL. Core tables include:

- users (user_id PK, name, email (unique), password_hash, is_admin BOOLEAN).
- farms (farm_id PK, name, latitude, longitude, area). [owner_id removed as sole authority - ownership now expressed via farm_collaborators; farms.created_by retained for audit]
- farm_collaborators (farm_id FK, user_id FK, role ENUM('owner','viewer'), invited_by FK->users, added_at) -
PRIMARY KEY (farm_id, user_id) [NEW].
- fields (field_id PK, farm_id FK->farms, name, polygon (geo), area).

- crops (crop_id PK, name).
- varieties (variety_id PK, crop_id FK, name, description).
- growth_stages (stage_id PK, crop_id FK, stage_name, days_after_plant).
- crop_instances (ci_id PK, field_id FK, variety_id FK, planting_date, harvest_date, current_stage_id FK, active

BOOLEAN).

- gateways (gw_id PK, field_id FK, model, status, last_seen).
- sensor_bases (base_id PK, gw_id FK, label).
- sensor_slots (slot_id PK, base_id FK, slot_number, sensor_type).
- sensor_readings (reading_id PK, slot_id FK, timestamp, value_float).
- weather_data (weather_id PK, field_id FK, timestamp, temperature, humidity, precipitation, wind, etc.).

- recommendations (rec_id PK, field_id FK, ci_id FK (optional), datetime, action_text, evidence_text,

source_refs, confidence_score).

- feedback (fb_id PK, rec_id FK, user_id FK, rating INT, comments TEXT, timestamp, followed_ynno BOOL,

outcome_ynno BOOL).

- knowledge_entries (kb_id PK, title, content, source, date_added, vector_embedding).
- alerts (alert_id PK, farm_id FK, field_id FK NULLABLE, alert_type, severity

ENUM('low','medium','high','critical'), message, source_ref, status, created_at, expires_at) [NEW].

- alert_recipients (alert_id FK, user_id FK, read_at NULLABLE, dismissed_at NULLABLE) - PRIMARY KEY

(alert_id, user_id) [NEW].

Indexes: sensor_readings(field_id, timestamp); recommendations(field_id); full-text on knowledge_entries(content); farm_collaborators(user_id) for fast "My Farms" lookups [NEW]; alert_recipients(user_id, read_at) for unread-count queries [NEW]; alerts(farm_id, created_at) [NEW].

Sample Row (farm_collaborators):

farm_id user_id role added_at

17 5 owner 2026-06-01 17 9 viewer 2026-06-15

Sample Row (alerts):

alert_id farm_id field_id alert_type severity status

301 17 88 sensor_out_of_range medium open 302 17 null severe_weather high open

6.1 Connectivity Data Additions [NEW]

Add logical persistence for gateway_connectivity_status (gw_id PK/FK, active_transport, wifi_ssid_hash or masked identifier, wifi_signal_dbm, cellular_signal, last_cloud_success_at, queued_records, queue_capacity_percent, fallback_reason, updated_at), connectivity_events (event_id PK, gw_id FK, event_type, from_transport, to_transport, reason, created_at), and provisioning_audit (audit_id PK, gw_id FK, initiated_by FK, method ENUM('ble','softap'), result, created_at).

Saved Wi-Fi passwords should not be stored in the application database. The cloud may store non-secret metadata such as a masked SSID label, priority, provisioning status, and last successful use. The actual credential is encrypted at rest on the controller using device-bound key material. If a remote encrypted credential envelope is introduced later, it must be encrypted specifically for the target device and never be readable by ordinary application services.

Sensor readings uploaded after an outage should include device_sequence and captured_at fields, with a uniqueness constraint such as (slot_id, device_sequence) or another idempotency key to prevent duplicate rows during retries.

7. AI Recommendation Engine (RAG Pipeline)

Unchanged from the base design (Knowledge Indexing, Context Builder, Prompt Template, LLM Call, Post-Processing, Feedback Loop). One addition: when the Recommendation Service classifies a generated recommendation as high-risk (e.g. disease probability above threshold), it now also emits a "high_risk_recommendation" event to the Alert Service so affected users are proactively notified rather than only seeing it if they open the AI Advisor page [NEW].

8. Hardware Integration

Unchanged from the base design (Controller/Gateway architecture, SensorBase and modules, firmware updates, sensor-to-field mapping). One addition: the Hardware Service now emits a "gateway_offline" event when a gateway's last_seen exceeds a configurable threshold (e.g. 6 hours), consumed by the Alert Service [NEW hook - see Sect. 12].

8.1 Resilient Connectivity [NEW]

Each controller supports Wi-Fi-first operation with the current 2G/GPRS module as automatic fallback. The firmware shall abstract transports behind a common network interface so future hardware can add Ethernet, LTE Cat 1 bis, LTE-M, NB-IoT, or another supported cellular module without changing farm workflows or MQTT payload semantics. The current implementation does not remove or alter the existing SIM workflow.

Provisioning: During physical setup, the controller enters a time-limited provisioning mode after a button press or authenticated claim. Preferred method is BLE because the phone can remain connected to its current network; protected SoftAP is the fallback. The session requires controller proof of possession, uses an authenticated/encrypted exchange, times out automatically, and clears temporary session material after completion. The phone sends SSID and password directly to the nearby controller. The controller encrypts the credential locally and returns only success/failure and non-secret network metadata.

Credential storage: Store a maximum configurable number of trusted networks (recommended 2-3) with priority, security type, last-success timestamp, and encrypted credential blob. Device-bound encryption keys should be held in secure hardware/secure storage where available. Logs, crash dumps, MQTT payloads, support bundles, and cloud APIs must redact passwords. A factory reset clears credentials and ownership material according to a documented recovery flow.

Connection selection: The controller tries trusted Wi-Fi networks in priority order, evaluating signal and completing an end-to-end cloud test rather than treating Wi-Fi association alone as success. After repeated failures it activates cellular. While on cellular it periodically probes Wi-Fi with backoff. Hysteresis and minimum dwell time prevent flapping. A manual diagnostic override may force Wi-Fi or cellular temporarily, but Automatic is the default production mode.

Store-and-forward: Every sensor record is written to a local persistent queue before or as part of transmission. Records contain original captured_at time, sensor/slot identifier, monotonic device sequence, payload version, retry count, and integrity information. After cloud acknowledgement, records may be safely removed. During recovery, critical events are sent first, recent telemetry next, and historical backlog in rate-limited batches. Queue capacity, retention policy, flash wear, corruption recovery, and power-loss safety must be tested.

Traffic policy: Critical alerts, gateway health, and security events receive highest priority. Normal telemetry may be compressed/batched. Historical backlog must not starve live messages. Large firmware/knowledge payloads should prefer Wi-Fi and require power/battery checks. Cellular usage counters should be exposed to control SIM cost.

9. API Specification

All APIs use HTTPS and JWT Bearer tokens. Authorization for farm-scoped endpoints is resolved against farm_collaborators (Owner vs Viewer), not solely from the JWT [UPDATED].

Authentication:

- POST /api/auth/register – Body: {name, email, password} -> 201 Created or error.
- POST /api/auth/login – Body: {email, password} -> 200 OK with {token, user_id}.
- GET /api/auth/refresh – refresh JWT.

Farms & Fields:

- GET /api/farms -> list of farms the user owns or is a viewer on, each annotated with the caller's role

[UPDATED].

- POST /api/farms – Body: {name, location, area} -> 201 Created; caller becomes Owner.
- GET /api/farms/{farm_id} -> farm details (incl. fields). 403 if caller has no farm_collaborators row.
- PUT /api/farms/{id} – update metadata. Owner only.
- DELETE /api/farms/{id} – Owner only.
- POST /api/farms/{id}/fields – add field to farm. Owner only.
- GET /api/fields/{field_id} -> field details. Owner or Viewer.
- PUT/DELETE /api/fields/{id} – Owner only.

Farm Collaboration [NEW]:

- GET /api/farms/{farm_id}/collaborators -> list of {user_id, name, email, role, added_at}. Owner only.
- POST /api/farms/{farm_id}/collaborators – Body: {email, role: "viewer"} -> adds existing user or creates

pending invite. Owner only.

- DELETE /api/farms/{farm_id}/collaborators/{user_id} -> revoke access. Owner only. Cannot remove the sole Owner.

Crops / Crop Instances / Hardware: unchanged from base design - see Sect. 6 for referenced tables. Write endpoints (crop assignment, gateway/sensor registration) are Owner only; read endpoints are Owner or Viewer.

Sensor Data (Internal):

- GET /api/fields/{field_id}/sensors?type=soil_moisture&from;=..&to;=.. -> timeseries readings. Owner or Viewer.

Weather:

- GET /api/weather/current?field_id=X, GET /api/weather/forecast?field_id=X. Owner or Viewer.

Recommendations:

- POST /api/recommendations/generate – Owner only (Viewers can read but not trigger generation)

[UPDATED].

- GET /api/recommendations?field_id=X, GET /api/recommendations/{rec_id} -> Owner or Viewer.

Feedback:

- POST /api/feedback – Owner only [UPDATED: Viewers are read-only and cannot submit feedback].
- GET /api/feedback?user_id=X -> own feedback history.

Knowledge:

- GET /api/knowledge/search?q=term, GET /api/knowledge/crops/{crop_id} -> open to any authenticated user.

Alerts [NEW]:

- GET /api/alerts?farm_id=X&status;=unread -> list alerts for a farm, scoped to the caller's accessible farms.
- GET /api/alerts?status=unread -> list unread alerts across all farms the caller can access (for the global notification badge).

- POST /api/alerts/{id}/read -> mark as read for the calling user.
- POST /api/alerts/{id}/dismiss -> mark as dismissed for the calling user.
- POST /api/alerts/{id}/read-all -> mark all of the caller's unread alerts as read.

Each endpoint returns appropriate HTTP codes. Requests without valid JWT get 401. Requests where the caller has no farm_collaborators row for the target farm get 403. Write attempts by a Viewer get 403. Bad requests get 400.

Example API call (cURL):

POST /api/farms/17/collaborators HTTP/1.1

Authorization: Bearer

Content-Type: application/json

```
{ "email": "helper@example.com", "role": "viewer" }
```

Response 201 Created:

```
{ "farm_id": 17, "user_id": 9, "role": "viewer", "added_at": "2026-07-14T10:00:00Z" }
```

9.1 Connectivity APIs [NEW]

All endpoints remain farm-scoped and Owner/Viewer authorization follows the existing RBAC model. Credential payloads are accepted only for an authenticated, short-lived local provisioning session and are not persisted as plaintext by the backend.

- POST /api/gateways/{gw_id}/provisioning-sessions - create a time-limited provisioning challenge. Owner only; requires proof of possession or an already claimed controller.
- GET /api/gateways/{gw_id}/connectivity - current transport, signal, last cloud success, queue depth, configured-network count, and fallback reason. Owner or Viewer; no secrets.
- POST /api/gateways/{gw_id}/connectivity/test - request a connection/cloud test. Owner only.
- PUT /api/gateways/{gw_id}/connectivity/mode - Body: {mode:"automatic"|"wifi"|"cellular", diagnostic_ttl_minutes}. Owner only; non-automatic modes should expire.
- DELETE /api/gateways/{gw_id}/wifi-networks/{network_id} - remove a saved network through a secure device command. Owner only.
- PUT /api/gateways/{gw_id}/wifi-networks/order - reprioritize saved networks. Owner only.
- POST /api/gateways/{gw_id}/sync - request backlog synchronization when connectivity and power policy permit. Owner only.

The BLE/SoftAP local provisioning channel may use a device-local protocol rather than a cloud REST endpoint. Cloud APIs should exchange session challenges, status, and audit results, not reusable plaintext passwords. Device-to-cloud uploads include idempotency keys and cloud acknowledgements to support safe retry.

10. Deployment Architecture

Infrastructure: Docker containers on Kubernetes. PostgreSQL (managed RDS/Aurora), EMQX for MQTT, Kafka cluster for streaming, DeepSeek accessed via API, Weather API with key in Secrets. The Alert Service [NEW] is deployed as its own replicated Deployment (>=2 replicas) since it sits on the critical path for time-sensitive notifications and consumes from multiple Kafka topics (sensor, weather, hardware, recommendation).

CI/CD: GitHub Actions/GitLab CI - test, build, deploy per service, including the new Collaboration and Alert services.

Monitoring: Centralized logs (ELK/EFK), Prometheus/Grafana metrics, Alertmanager - note: Alertmanager (infra-level) is separate from the product-level Alert Service; both should be documented distinctly to avoid confusion during on-call.

Backup & Recovery: Automated daily DB snapshots (now including farm_collaborators, alerts, alert_recipients tables); Kafka retention >=7 days.

10.1 Connectivity Deployment Additions [NEW]

The MQTT broker and ingestion APIs must accept both Wi-Fi and cellular traffic without assigning different device identities. Every controller uses one unique certificate/key identity regardless of active transport. Connectivity metrics are exported to Prometheus/Grafana: active transport distribution, Wi-Fi failure rate, cellular fallback duration, offline queue depth, synchronization lag, failed provisioning attempts, and per-device cellular data usage where available.

Firmware update policy should support signed OTA packages, resumable downloads, rollback, and Wi-Fi-preferred scheduling. Cellular OTA may be allowed only for urgent security fixes or explicit Owner approval. The hardware design should keep the cellular subsystem modular so a future 4G-capable module can replace the 2G modem without redesigning the complete controller.

11. Security

- Authentication: JWT Bearer tokens. Passwords hashed (bcrypt). JWT payload carries only user_id and

is_admin - no farm-specific role is embedded, preventing stale-permission tokens after a Viewer is removed [UPDATED].

- Role-Based Access Control (RBAC) [UPDATED - see full model in Sect. 13]: Two tiers - platform-level

(Admin vs regular User) and farm-level (Owner vs Viewer, resolved per request from farm_collaborators). Admins manage the knowledge base and platform operations; they do not automatically gain farm-level access to individual farms' data.

- Transport Security: HTTPS (TLS 1.2+), MQTT over TLS, Kafka SASL encryption.
- Encryption: PostgreSQL data at rest encrypted; sensitive data (passwords, API keys) encrypted in storage.
- Secrets Management: API keys in environment variables or a vault (AWS Secrets Manager).

- Input Validation: All inputs validated and sanitized; sensor values range-checked.

- Audit Logging: Critical actions - including collaborator additions/removals [NEW] and alert dismissals [NEW]

- are logged with user ID and timestamp.

- Data Privacy: Minimal PII stored. Removing a Viewer immediately revokes their read access on next request

(enforced server-side, not just hidden client-side) [NEW]. Compliance with local data laws (data deletion on request, exportable data).

11.1 Connectivity and Credential Security [NEW]

- Provisioning requires physical proximity or controller proof of possession, a short-lived session, replay protection, and authenticated encryption.

- Wi-Fi passwords are write-only. They are never returned to the UI, included in telemetry, written to normal logs, or exposed to Viewers.

- Credentials are encrypted at rest on the controller with device-bound keys; factory reset and ownership-transfer procedures clear or rotate them.

- The controller validates server certificates and uses MQTT/HTTPS over TLS on both transports. Wi-Fi availability must never cause TLS downgrade or certificate bypass.

- Device identity is unique per controller. Fleet-wide shared private keys are prohibited.
- Rate limits and lockouts protect provisioning attempts. Provisioning mode automatically expires and should require a physical action for recovery after repeated failures.

- Saved networks must use supported security modes; automatic association to open/public Wi-Fi is disabled.

- Connectivity configuration changes, diagnostic overrides, resets, and failed provisioning attempts are audit logged.

12. Alert System [NEW SECTION]

12.1 Purpose

Farmers need timely, actionable warnings without having to actively check the dashboard. The Alert System consolidates events from four upstream sources into a single, farm-scoped notification stream shared by the Owner and all Viewers of that farm.

12.2 Alert Sources and Types

- Weather-triggered: `severe_weather` (e.g. rain probability >80%, extreme heat/cold, high wind) - raised by the Weather Service.
- Sensor-triggered: `sensor_out_of_range` (reading outside safe band for the crop's growth stage), `sensor_no_data` (no reading received within expected interval) - raised by the Sensor Data Service.
- Hardware-triggered: `gateway_offline` (no heartbeat within threshold), `battery_low` - raised by the Hardware Service.
- AI-triggered: `high_risk_recommendation` (disease/pest risk above confidence threshold identified by the RAG pipeline) - raised by the Recommendation Service.

12.3 Severity and Deduplication

Each alert is classified low/medium/high/critical based on source-specific rules (e.g. rain probability 50-79% = medium, >=80% = high; gateway offline 6-24h = medium, >24h = high). Before creating a new alert, the Alert Service checks for an existing open alert of the same (farm_id, field_id, alert_type) to avoid flooding recipients with duplicates; instead it updates the existing alert's severity/timestamp if the condition persists or worsens.

12.4 Delivery and Lifecycle

Alerts are persisted to alerts and fanned out via alert_recipients to every current member of farm_collaborators for that farm - Owner and Viewers alike, since risk information is relevant regardless of write access. Delivery channels: in-app dashboard badge and Alerts page (real-time via WebSocket/SSE or short-poll); email notification if enabled in user preferences (future work notes push/SMS as long-term enhancements). Each recipient independently tracks read_at and dismissed_at, so one Viewer dismissing an alert does not affect another recipient's view.

12.5 Data Model

See Sect. 6 for the alerts and alert_recipients tables. See Sect. 9 for the alert read/dismiss endpoints.

12.6 Connectivity Alert Types [NEW]

The Alert Service additionally consumes connectivity events emitted by the Hardware Service:

- `wifi_connection_failed` - repeated failure of all configured Wi-Fi networks. Normally informational when cellular fallback succeeds.
- `cellular_fallback_activated` - controller has moved from Wi-Fi to cellular. Record as an event and optionally notify according to user preference; do not create repeated alerts during ordinary brief outages.
 - `cellular_signal_weak` - sustained signal below a configured threshold while cellular is the active path.
- `all_networks_unavailable` - neither Wi-Fi nor cellular can reach the cloud for longer than the configured grace period.
 - `offline_buffer_nearly_full` - durable queue exceeds a capacity threshold and data-loss risk is increasing.
 - `offline_data_synchronized` - backlog has been acknowledged by the cloud. Usually a recovery notification rather than a warning.
 - `provisioning_failed` - repeated or security-relevant setup failure; visible to the Owner only.

Deduplication uses (farm_id, field_id, gateway_id, alert_type). Short transitions are suppressed through grace periods. Severity should reflect operational impact: successful cellular fallback is low/informational; sustained total outage is medium/high; buffer-nearly-full is high; buffer-full or detected data loss is critical. Owner and Viewers receive operational outage alerts under the existing recipient model, while credential/provisioning security alerts are Owner-only.

13. Role-Based Access & Farm Collaboration [NEW SECTION]

13.1 Two-Tier Role Model

SPECTRON AgriAssist uses two independent role tiers:

- Platform tier: is_admin flag on users. Admins manage the shared Knowledge Base and platform operations;

this does not grant them implicit access to any individual farm's data.

- Farm tier: role stored per (farm, user) pair in farm_collaborators - either owner or viewer. This is what governs access to a specific farm's fields, hardware, sensor data, recommendations, and alerts.

13.2 Owner Permissions

An Owner can: create/edit/delete the farm and its fields; register and configure hardware (gateways, sensor bases, sensor modules); assign crops and update growth stages; trigger AI recommendation generation; submit feedback; invite and remove Viewers; and transfer ownership (future enhancement - not in initial scope, since a farm currently requires exactly one Owner at all times).

13.3 Viewer Permissions

A Viewer can: view farm/field details, sensor graphs, weather, existing recommendations and their evidence, recommendation history, and alerts for that farm. A Viewer cannot: edit farm/field/hardware data, trigger recommendation generation, submit feedback, or manage collaborators. All Viewer-facing pages render the same data as the Owner view but with edit controls hidden and write API calls blocked server-side (403), not just hidden in the UI.

13.4 Invitation Flow

1. Owner submits an email on the "Manage Access" page.
2. If the email belongs to an existing platform user, a farm_collaborators row (role=viewer) is created

immediately.

3. If not, an invitation token is generated and emailed; upon registration with that email, the invite is

auto-redeemed.

4. The new Viewer sees the farm appear in "My Farms" with a "Viewer" badge on next login or token refresh.

13.5 Enforcement

All farm-scoped service calls resolve the caller's role via a shared authorization check against farm_collaborators (not the JWT), so revoking a Viewer takes effect immediately on their very next request rather than only after their token expires. This check is implemented once as shared middleware/library code and reused across Farm, Field, Hardware, CropInstance, Recommendation, Feedback, and Alert services to avoid inconsistent enforcement.

13.6 Connectivity Permissions [NEW]

Owners can provision, add, remove, reprioritize, and test Wi-Fi networks; change Automatic/diagnostic connectivity mode; request synchronization; and view connectivity diagnostics. Viewers can view current transport, signal quality, last contact, and applicable farm alerts but cannot see passwords, network secrets, provisioning tokens, or issue configuration commands. Server-side authorization is required for every cloud command; hiding controls in the UI is insufficient.

14. Testing, QA, and Rollout

- Unit Tests: Each service >80% coverage, including new Collaboration and Alert services [UPDATED].
- Integration Tests: Multi-service flows - including "invite viewer -> viewer reads farm data -> owner revokes

-> viewer receives 403" and "sensor out-of-range -> alert created -> both owner and viewer see it" [NEW cases].

- Hardware-in-the-loop: Staging IoT environment for MQTT/Kafka pipeline validation, including gateway_offline

alert triggering after simulated disconnect [NEW case].

- UI Testing: Automated Selenium/Cypress tests for login, farm creation, receiving a recommendation,

inviting/removing a viewer, and alert read/dismiss flows [UPDATED].

- Performance Testing: Load test Kafka ingestion; stress test LLM calls; load test Alert Service fan-out for farms

with many collaborators [NEW].

- Penetration Testing: OWASP checks, including explicit tests that Viewers cannot access write endpoints via

direct API calls [NEW focus].

- Beta Rollout: Pilot with a small group of farmers (5 fields, different regions); include at least one

multi-collaborator farm in the pilot to validate the sharing flow [UPDATED].

14.1 Connectivity Test Additions [NEW]

- Unit Tests: network state transitions, retry/backoff, hysteresis, credential redaction, queue ordering, acknowledgement, and idempotency.

• Integration Tests: Wi-Fi online -> Wi-Fi loss -> cellular fallback -> Wi-Fi recovery; both networks down -> local buffering -> reconnect -> ordered synchronization without duplication.

- Security Tests: unauthorized provisioning, replay, expired session, wrong proof-of-possession code, log/telemetry secret leakage, Viewer write attempts, and factory-reset behavior.

• Hardware-in-the-loop: weak Wi-Fi, wrong password, captive portal/no internet, DNS failure, TLS failure, SIM missing, low cellular signal, power loss during queue write, flash-full condition, and modem reset.

- Load/Endurance Tests: long offline periods, large backlog, live traffic during catch-up, flash wear, cellular-cost limits, and many controllers reconnecting simultaneously.

- UX/UAT: configure during Add Controller, skip and configure later, change router password, use mobile hotspot, understand fallback status, and recover a replaced router without technical assistance.

• Pilot: include Wi-Fi-dense urban/peri-urban sites and remote cellular-only farms; measure transport uptime, fallback frequency, data-loss rate, queue age, provisioning success, and SIM data consumption.

15. Maintenance & Future Work

- Knowledge Base Updates: New crop guides or pest outbreaks added by admins as they arise.

- Software Updates: Regular dependency/security patches; feature sprints (satellite imagery, CV-based pest

ID).

- Scaling Out: More Kafka brokers, DB replicas, Kubernetes pods as usage grows.

- Data Retention: Archive old sensor data as needed.

• Alerting Enhancements [NEW]: SMS/push notification channels; per-alert-type notification preferences;

configurable per-farm thresholds (currently global defaults).

• Access Control Enhancements [NEW]: Ownership transfer between users; a third "Editor" role between

Owner and Viewer (can trigger recommendations/feedback but not manage hardware or collaborators) if farmer feedback indicates a need.

- Other Long-Term Enhancements: Drone integration, yield prediction models, government subsidy integration,

mobile app, AI model fine-tuning, offline mode.

15.1 Connectivity Future Work [NEW]

- Replace the fixed 2G-only assumption with a modular cellular interface. Evaluate locally supported LTE Cat 1 bis, LTE-M, NB-IoT, or 4G/2G dual-mode hardware before production scale-up.
 - Optional Ethernet for fixed installations and service centers.
 - Remote encrypted credential envelopes only if needed, using per-device public-key encryption and strict secret-handling controls.
 - Per-farm cellular budgets, data-usage reports, and policies controlling backlog/OTA over cellular.
 - Mobile-app assisted BLE provisioning and guided signal/site survey.
- Connectivity analytics to recommend controller relocation, antenna changes, or network configuration improvements.

16. Implementation Roadmap

A high-level project plan spanning ~9-12 months. The Alert System and Collaboration/RBAC work are folded into the existing Backend Core and Frontend milestones rather than treated as separate phases, since both touch nearly every existing service.

- Requirements & Design (Aug 2026): Finalize requirements, UI mockups - now including Manage Access and Alerts pages (UI/UX Designer – 5 pw, +1 for new pages).
- Frontend Development (Sep-Nov 2026): Implement pages/components including Manage Access and Alerts pages (Front-end Engineers – 14 pw, +2).
- Backend Core (Aug-Oct 2026): Auth, Farm/Field services, DB schema, Collaboration Service, Alert Service (Back-end Engineers – 14 pw, +4). Kubernetes/CI/CD (DevOps – 4 pw).
- Hardware Integration (Aug-Oct 2026): EMQX, Kafka, controller firmware, gateway_offline event emission (Embedded Engineer – 8 pw).
- Sensor Data & Weather (Oct-Nov 2026): Sensor ingestion, weather API, out-of-range/severe-weather event emission (Back-end – 5 pw, +1).
- Knowledge Base (Sep-Dec 2026): Scrape/import DOA/IRRI docs, build RAG index (Data Engineer – 6 pw).
- AI Integration (Oct-Dec 2026): Context builder, prompt templates, DeepSeek interface, high-risk recommendation event emission (AI Engineer – 8 pw).
- UI/UX Polishing (Nov-Dec 2026): Accessibility audit, multi-language support (UI/UX & Front-end – 4 pw).
- Testing & QA (Dec 2026): System/UAT including RBAC and alert test cases (QA Engineer – 7 pw, +1).
- Deployment & Pilot (Jan 2027): Deploy to production, train pilot farmers including multi-collaborator farm, monitoring setup (DevOps & Support – 6 pw).

Resource Estimates (updated):

Role Effort (person-weeks) Responsibility

Project Manager 10 pw Oversight, coordination

UX/UI Designer 7 pw Page mockups, accessibility, branding,
new pages

Frontend Developer(s) 14 pw App, localization, Manage Access &
Alerts UI

Backend Developer(s) 16 pw Go microservices incl. Collaboration &
Alert Service

Embedded Engineer 8 pw Sensor/Gateway firmware and MQ
setup

AI/Data Engineer 6 pw Knowledge crawling, RAG indexing

AI/ML Engineer 8 pw Prompt design, LLM integration

DevOps/Infra 6 pw CI/CD, K8s, Kafka, EMQX, monitoring

QA/Test Engineer 7 pw Automated tests, RBAC & alert test

cases

Technical Writer 4 pw Documentation, manuals

(Total ~86 person-weeks over ~6 developers plus support staff, spread over ~9 months.)

16.1 Connectivity Work Items [NEW]

Connectivity work is integrated into existing milestones and does not alter the farm/crop workflows:

- Requirements & Design (Aug 2026): finalize state machine, provisioning UX, credential threat model, queue retention, and connectivity alert thresholds.
- Frontend Development (Sep-Nov 2026): optional Wi-Fi step in Add Controller; Configure Connectivity page; transport/signal/queue indicators; Owner-only controls.
- Backend Core (Aug-Oct 2026): connectivity status APIs, provisioning-session metadata, audit events, idempotent backlog ingestion, Alert Service event handling.
- Hardware Integration (Aug-Oct 2026): Wi-Fi driver, cellular fallback adapter, BLE/SoftAP provisioning, encrypted credential store, persistent queue, transport health tests, and connectivity event emission.
- Testing & QA (Dec 2026): failover, offline recovery, security, flash endurance, and field UAT scenarios.
- Deployment & Pilot (Jan 2027): pilot both Wi-Fi-preferred and cellular-only sites and collect uptime/cost metrics.

Resource impact: add firmware/embedded effort for network abstraction, provisioning and durable queue; backend effort for connectivity status, idempotent synchronization and alerts; frontend effort for setup/configuration UX; and security/QA effort for credential and failover testing. Exact person-week changes should be estimated after the cellular module and controller MCU/storage constraints are confirmed.

17. References

- IBM on RAG: Retrieval augmented generation is an architecture for optimizing AI model performance by connecting it with external knowledge bases.
- DOA Sri Lanka: Official rice cultivation statistics from RRD; soil/pH guidelines from DOA.
- Atlassian (Microservices vs Monolith): A microservices architecture is a collection of smaller, independently deployable services.
- EMQX Blog (MQTT+Kafka): Kafka is a distributed streaming platform for large-scale, real-time data streaming and processing; combining MQTT with Kafka provides reliable connectivity and efficient data exchange.
- DOA Safe Portal: Guidelines on site-specific fertilization.
- Additional agronomic references: IRRI, FAO, and CABI Plantwise.

(All citations correspond to authoritative sources from the original architecture document; this revision adds no new external citations, only internal design additions for Alerts and RBAC.)

18. Resilient Controller Connectivity (Wi-Fi + Cellular Fallback) [NEW]

18.1 Decision Summary

SPECTRON controllers use Wi-Fi as the preferred transport whenever a trusted configured network and the cloud service are reachable. The existing 2G/GPRS SIM module remains an automatic fallback, preserving current deployments and workflows. If neither path works, durable local store-and-forward prevents loss of sensor readings. This design changes the transport layer only; farm, field, crop, sensor, recommendation, alert, and RBAC workflows remain unchanged.

18.2 User Experience Decision

Wi-Fi configuration is offered during Add Controller as an optional step and is also available after addition from Configure Connectivity. This combined approach is preferred over forcing credentials at only one point: users with credentials complete setup immediately, while installers in cellular-only locations are not blocked. The Wi-Fi password is always treated as a write-only secret.

18.3 Operational Acceptance Criteria

- A controller continues publishing through cellular after sustained Wi-Fi failure without user intervention.
- A brief Wi-Fi interruption does not cause repeated transport flapping or alert flooding.
 - A controller stores readings safely during total outage and uploads them later without duplicates and with original timestamps.
 - Live critical messages are not delayed by historical backlog synchronization.
- Removing or changing a saved Wi-Fi network requires Owner authorization and leaves an audit record.
 - Viewers can understand controller status but cannot access credentials or configuration commands.
- No plaintext Wi-Fi password appears in cloud storage, logs, telemetry, analytics, support exports, or UI read responses.
 - The cellular hardware can be upgraded in a later revision without changing the application-level device identity or MQTT/API contracts.